



MODULE 11.2 JAVA FX

**ELVIA GISELLA ADAIR
STUDENT**

PROFESSOR LEWIS

MARCH 15, 2026

JAVAFX HBOX AND BORDERPANE

JavaFX is a modern framework used for building graphical user interfaces (GUIs) in Java applications. It provides several layout plans that help developers organize user interface components in a structured and responsive way. Layout panes control how elements such as buttons, labels, and text fields appear and behave within a window. Without these layout managers, it would be difficult to design interfaces that adapt properly when a window is resized or when components are added or removed. Two commonly used JavaFX layout panes are HBox and BorderPane. These layout containers help developers organize graphical elements efficiently and create well-structured user interfaces.

JAVAFX HBOX

The **HBox** layout pane arranges its child nodes in a single horizontal row. Each component placed in an HBox appears next to the previous one from left to right. This layout is useful when designing toolbars, button groups, navigation menus, or forms where elements should be aligned horizontally. HBox automatically manages spacing, alignment, and resizing behavior. Developers can specify the spacing between elements, alignment within the container, and padding around the edges. The layout also adjusts components if the window size changes. By default, HBox resizes child nodes to their preferred width and can expand them vertically depending on its configuration. One advantage of using HBox is simplicity. Developers can quickly place multiple controls side by side without manually calculating positions. HBox is commonly used for navigation bars, rows of buttons, or input forms where labels and text fields appear side by side.

Example:

Below is a simple example demonstrating how an HBox can be used to arrange buttons horizontally.

```
HBox hbox = new HBox(10); // spacing of 10 pixels
```

```
Button button1 = new Button("Home");
```

```
Button button2 = new Button("About");
```

```
Button button3 = new Button("Contact");
```

```
hbox.getChildren().addAll(button1, button2, button3);
```

In this example, three buttons are added to the HBox container. The number **10** indicates the spacing between each component. The result is a horizontal row of buttons that automatically adjusts based on the window size.

HBox is frequently combined with other layout panes such as VBox or BorderPane to create more complex interfaces. For example, an HBox may be placed inside the top section of another layout to create a navigation bar.

JAVAFX BORDERPANE

Another important JavaFX layout container is **BorderPane**. Unlike HBox, which only organizes elements in a single row, BorderPane divides the layout into five regions: **top, bottom, left, right, and center**.

Each region can contain a node or another layout pane. The center region typically holds the main content of the application, while the other areas are often used for navigation menus, toolbars, or status bars. The layout automatically adjusts the size of each region depending on the available space.

BorderPane is particularly useful for desktop-style applications that divide the interface into sections. For example:

- The top region may contain a menu bar or toolbar.
- The left or right regions may contain navigation panels.
- The bottom region may contain status information.
- The center region is usually reserved for the primary content area.

One of the main advantages of `BorderPane` is flexibility. Developers do not have to use all five regions. Any unused region simply does not occupy space in the layout.

Example:

Below is a simple example of a `BorderPane` layout.

```
BorderPane borderPane = new BorderPane();

Label topLabel = new Label("Application Header");
Button leftButton = new Button("Menu");
Label centerContent = new Label("Main Content Area");
Label bottomLabel = new Label("Status Bar");

borderPane.setTop(topLabel);
borderPane.setLeft(leftButton);
borderPane.setCenter(centerContent);
borderPane.setBottom(bottomLabel);
```

In this example, the `BorderPane` organizes elements into different regions of the interface. The header appears at the top, the menu button appears on the left side, and the main content appears in the center.

Developers often combine `BorderPane` with other layout panes such as `HBox` or `VBox`. For instance, an `HBox` containing multiple buttons may be placed in the top region to create a toolbar. This combination allows for more advanced and flexible user interface designs.

Comparison and Practical Use

Both `HBox` and `BorderPane` are essential layout tools in JavaFX development.

`HBox` focuses on organizing components horizontally, making it ideal for simple rows of controls. `BorderPane`, on the other hand, is designed for larger application layouts where different sections of the interface must be clearly separated.

In practice, developers frequently combine these layouts. For example, a `BorderPane` may serve as the main layout for the application window, while `HBox` containers are used within specific sections such as toolbars or button groups. This modular approach allows developers to create clean, organized interfaces that are easy to maintain and modify.

JavaFX layout panes play an important role in designing responsive, well-organized graphical user interfaces. The `HBox` layout allows developers to arrange components in a horizontal row, making it ideal for toolbars and grouped controls. The `Border Pane` layout divides the interface into five distinct regions, providing a flexible structure for more complex applications. By combining these layout panes, developers can build professional desktop applications with clear structure and adaptability. Understanding how these layout containers work is essential for any Java developer working with JavaFX

Example Java Program File (Submit Separately)

Save as **JavaFXLayoutExample.java**

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.BorderPane;
import javafx.scene.layout.HBox;
import javafx.stage.Stage;

/*
 * JavaFXLayoutExample.java
 * Demonstrates the use of HBox and BorderPane layout panes.
 * Author: Gisella Adair
 */

public class JavaFXLayoutExample extends Application {
```

@Override

```
public void start(Stage primaryStage) {
```

```
    // Create an HBox with buttons
```

```
    HBox hbox = new HBox(10);
```

```
    Button home = new Button("Home");
```

```
    Button about = new Button("About");
```

```
    Button contact = new Button("Contact");
```

```
    hbox.getChildren().addAll(home, about, contact);
```

```
    // Create BorderPane layout
```

```
    BorderPane borderPane = new BorderPane();
```

```
    Label header = new Label("JavaFX Layout Example");
```

```
    Label content = new Label("This is the main content area.");
```

```
    Label footer = new Label("Status: Running");
```

```
    borderPane.setTop(header);
```

```
    borderPane.setCenter(content);
```

```
    borderPane.setBottom(footer);
```

```
    borderPane.setLeft(hbox);
```

```
    Scene scene = new Scene(borderPane, 400, 250);
```

```
    primaryStage.setTitle("JavaFX Layout Demo");
```

```
    primaryStage.setScene(scene);
```

```
    primaryStage.show();
```

```
}
```

```
public static void main(String[] args) {
```

```
    launch(args);  
}  
}
```

REFERENCES

1. https://docs.oracle.com/javafx/2/layout/builtin_layouts.htm
2. <https://docs.oracle.com/javase/8/javafx/api/javafx/scene/layout/HBox.html>
3. <https://docs.oracle.com/javase/8/javafx/api/javafx/scene/layout/BorderPane.html>
4. <https://examples.javacodegeeks.com/java-development/desktop-java/javafx/javafx-borderpane-example/>